

GRAND CANYON UNIVERSITY™

# DadPath

Andrew Rodriguez

CST-451 Capstone Project Final Architecture & Design

Grand Canyon University

Instructor: Professor Scott Webster

Revision: 1

Date: 26 April 2026

© 2020. Grand Canyon University. All Rights Reserved.

## ABSTRACT

This document presents the final architectural plan for the DadPath application during the design phase. It outlines the proposed system structure, including the frontend, backend, and database components, and explains how these elements will interact to form a scalable and secure full-stack web application. The system is designed as an Angular single-page application (SPA) for the frontend and a Spring Boot backend that communicates via RESTful APIs, using JSON Web Token (JWT) authentication to ensure secure access control.

The architectural design consists of a layered structure that includes controllers, services, repositories, and data transfer objects (DTOs). This separation of concerns promotes maintainability and scalability. The design also specifies the integration of a relational database for persistent data storage and details the data flow across system components. Supporting architectural diagrams and design models illustrate system organization and component interactions.

This document serves as the definitive design blueprint for implementation, ensuring the system adheres to modern software engineering principles and industry best practices. It provides a structured foundation for development, testing, and future expansion of the DadPath platform.


## History and Signoff Sheet

### Change Record

| Date | Author | Revision Notes |
|---|---|---|
| 7 Apr 26’ | Andrew | Initial draft for review/discussion |
| 16 April 26’ | Andrew | Update initial draft |
| 24 April 26’ | Andrew | Edit formatting / clean-up |

**Overall Instructor Feedback/Comments**

**Overall Instructor Feedback/Comments**

**Integrated Instructor Feedback into Project Documentation**

☐ Yes ☐ No

© 2020. Grand Canyon University. All Rights Reserved.

# TABLE OF CONTENTS

**DESIGN OVERVIEW** ..... **4**
**DETAILED HIGH-LEVEL SOLUTION DESIGN** ..... **5**
**DETAILED TECHNICAL DESIGN** ..... **7**
**APPENDIX A – TECHNICAL ISSUE AND RISK LOG** ..... **19**
**APPENDIX B – REFERENCES** ..... **20**
**APPENDIX C – EXTERNAL RESOURCES** ..... **21**

© 2020. Grand Canyon University. All Rights Reserved. 

## Design Introduction

The proposed DadPath solution uses a layered full-stack architecture to create a scalable, maintainable, and user-friendly platform for expecting fathers. The system includes an Angular single-page application (SPA) frontend that communicates with a Spring Boot REST API, which in turn interacts with a PostgreSQL database. This setup ensures a clear separation between the presentation layer, business logic, and data storage, enabling each component to develop independently while preserving overall system integrity.

At a high level, the frontend handles user interaction, navigation, and communication with backend services. The backend manages incoming requests, enforces business rules, handles authentication and data management, and is organized into separate layers (controller, service, repository, and entity/DTO). This organization promotes modularity, reusability, and ease of testing. Secure communication between the client and server uses JSON Web Tokens (JWT), enabling stateless authentication and enhancing scalability.

This design supports the expected user experience, including secure login, organized navigation, and future features such as memory timelines, preparation checklists, and informational resources. Supporting artifacts, including UI mockups, flowcharts, and process diagrams, are included in the following sections to demonstrate system behavior and user interaction. These materials give stakeholders a clear understanding of the proposed solution and its functionality before full technical implementation.

| Deliverable Acceptance Log | | | | | |
|---|---|---|---|---|---|
| ID | Deliverable Description | Comments | Evaluator (internal or external as applicable) | Status | Date of Decision |
| 1 | API Design Documentation | Defines database fields, types, and constraints for all entities | Internal | Approved | |
| 2 | Deployment Configuration | Full-stack implementation (Angular + Spring Boot) | Internal | Planned | |
| 3 | User Documentation | Environment setup for hosting (Vercel, Railway) | Internal | Planned | |
| 4 | Data Dictionary | Defines database fields, types, and constraints for all entities | Internal | Approved | |
| 5 | Source Code Repository | May be published separately for frontend or third-party integration | Internal | Pending | |

© 2020. Grand Canyon University. All Rights Reserved.

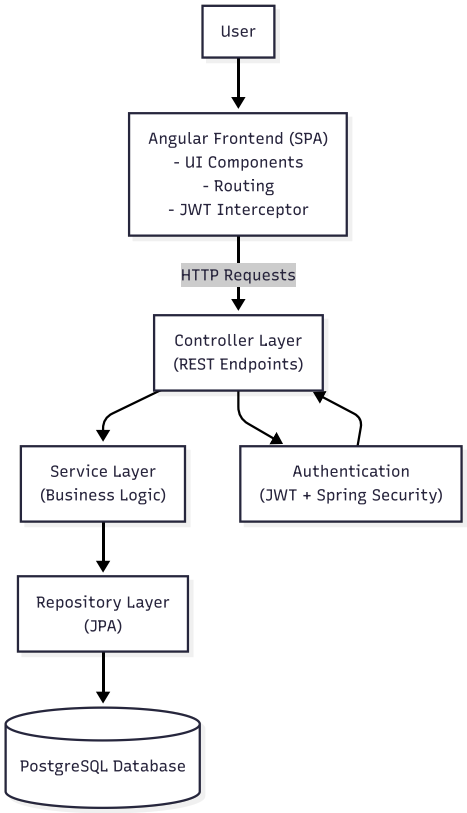
## Detailed High-Level Solution Design

| Proof of Concepts | | |
|---|---|---|
| **Description** | **Rationale** | **Results** |
| 1. JWT Authentication with Spring Security | To verify secure, stateless authentication and proper user authorization without server-side sessions | Successfully implemented login, token generation, and protected endpoints. JWT proved secure and scalable |
| 2 - Angular ↔ Spring Boot API Integration | To ensure seamless communication between frontend and backend using REST APIs | HTTP requests worked correctly, JWT interceptor attached tokens, and protected routes were enforced |
| 3 - PostgreSQL with JPA/Hibernate + Connection Pooling | To validate efficient data persistence and avoid performance issues from multiple connections | CRUD operations successful, stable entity mapping, and improved performance using connection pooling |
| 4 - Deployment (Vercel + Railway) | To confirm the system can run in a production-like environment with proper configurations | Application deployed successfully, environment variables worked, and services communicated correctly |
| 5 - Password Security using BCrypt | To ensure user credentials are securely stored and protected from breaches | Passwords are hashed securely, preventing plaintext storage and improving overall system security |

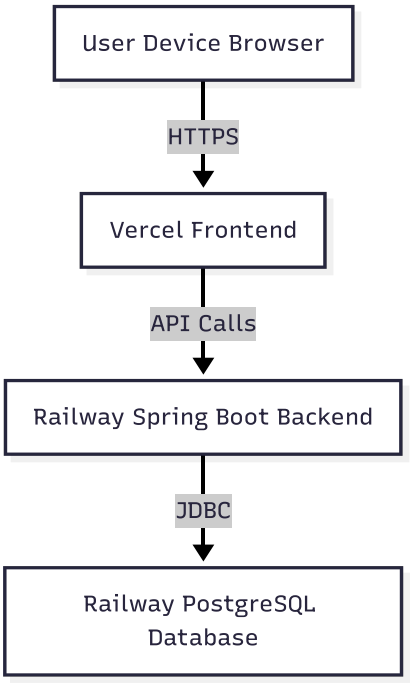
| Hardware and Software Technologies |
|---|
| 1 – Spring Boot + Spring Security |
| 2 – Angular + Angular Material + Bootstrap |
| 3 – PostgreSQL + JPA/Hibernate + HikariCP |
| 4 – JWT + BCrypt |
| 5 – Vercel + Railway (Deployment) |

© 2020. Grand Canyon University. All Rights Reserved.

**Logical Solution Design:**




**Physical Solution Design:**




© 2020. Grand Canyon University. All Rights Reserved.

## Detailed Technical Design

### General Technical Approach:

The design approach for DadPath involved iterative planning and refinement, with a primary focus on the core system architecture and security. Early brainstorming sessions emphasized the importance of a scalable backend, secure authentication, and a user-friendly interface. These factors guided the choice of technologies and shaped the overall system design.

### Key Technical Design Decisions:

Spring Boot (Backend Framework)

- **Purpose:** Provides REST API, business logic, and layered architecture.
- **Why chosen:** Strong ecosystem, rapid development, built-in support for security and dependency injection.
- **POC:** Implemented authentication endpoints and verified controller–service–repository structure.

Spring Security + JWT (Authentication)

- **Purpose:** Secure user authentication and authorization.
- **Why chosen:** Enables stateless authentication, improving scalability and simplifying session management.
- **POC:** Successfully implemented login, token generation, and protected routes.

Angular (Frontend Framework)

- **Purpose:** Builds the single-page application (SPA) interface.
- **Why chosen:** Component-based architecture, routing support, and seamless API integration.
- **POC:** Verified frontend-backend communication using HTTP client and JWT interceptor.

PostgreSQL + JPA/Hibernate + HikariCP

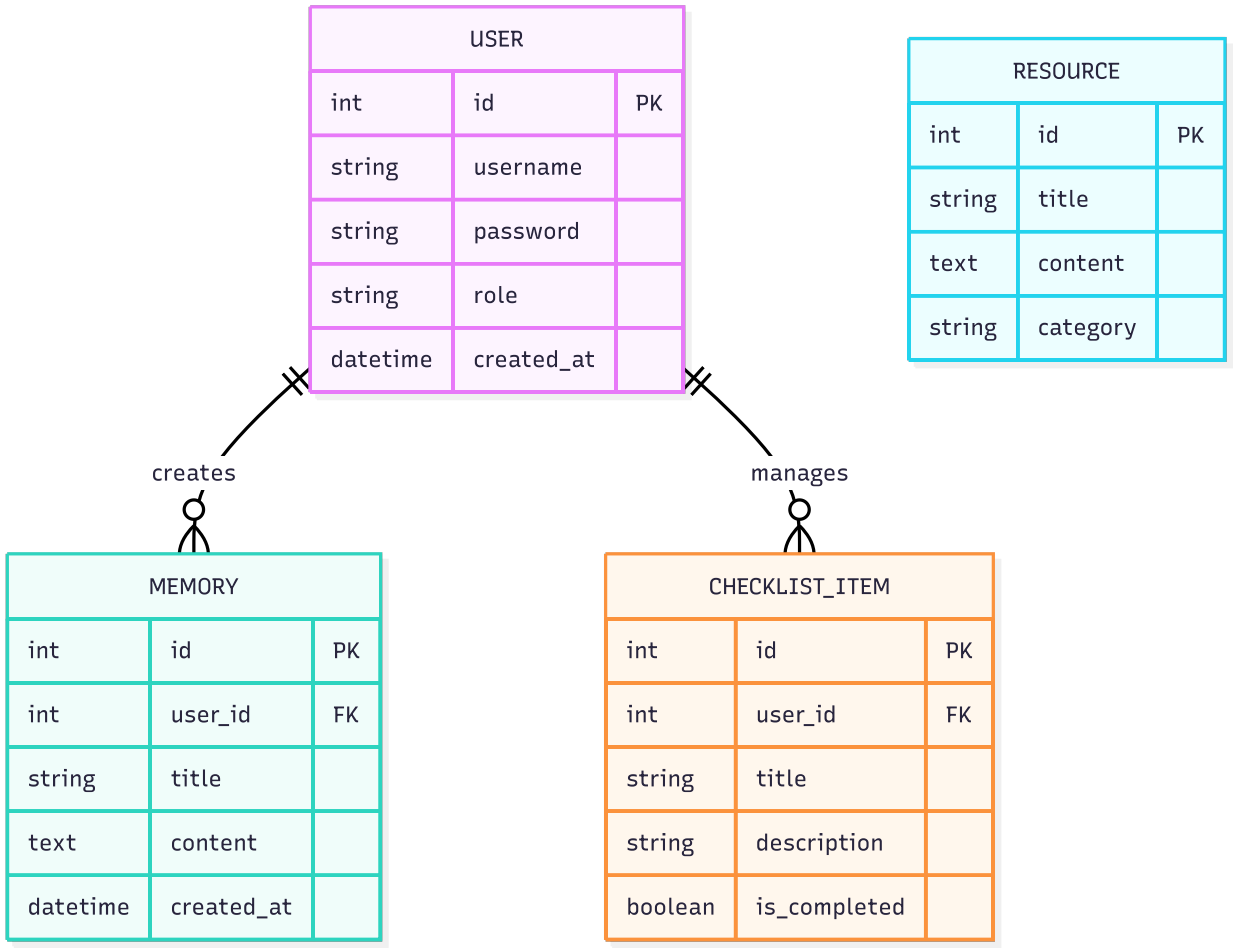
- **Purpose:** Data persistence and database interaction.
- **Why chosen:** PostgreSQL provides reliability; JPA/Hibernate simplifies ORM; HikariCP ensures efficient connection pooling.
- **POC:** Confirmed CRUD operations, stable entity mapping, and improved performance with connection pooling.

Vercel + Railway (Deployment Platforms)

- **Purpose:** Host frontend (Vercel) and backend/database (Railway).
- **Why chosen:** Simple deployment workflow, environment variable support, and scalability.
- **POC:** Successfully deployed and validated communication between services.

© 2020. Grand Canyon University. All Rights Reserved.

**Database ER Diagram:**




© 2020. Grand Canyon University. All Rights Reserved.

**Database DDL Scripts:**

```
-- ========================
-- USER TABLE
-- ========================
CREATE TABLE users (
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
username VARCHAR(50) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
role VARCHAR(20) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ========================
-- MEMORY TABLE
-- ========================
CREATE TABLE memory (
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
user_id INT NOT NULL,
title VARCHAR(100) NOT NULL,
content TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


CONSTRAINT fk_memory_user
FOREIGN KEY (user_id)
REFERENCES users(id)
ON DELETE CASCADE
);

-- ========================
-- CHECKLIST ITEM TABLE
-- ========================
CREATE TABLE checklist_item (
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
user_id INT NOT NULL,
title VARCHAR(100) NOT NULL,
description TEXT,
is_completed BOOLEAN DEFAULT FALSE,

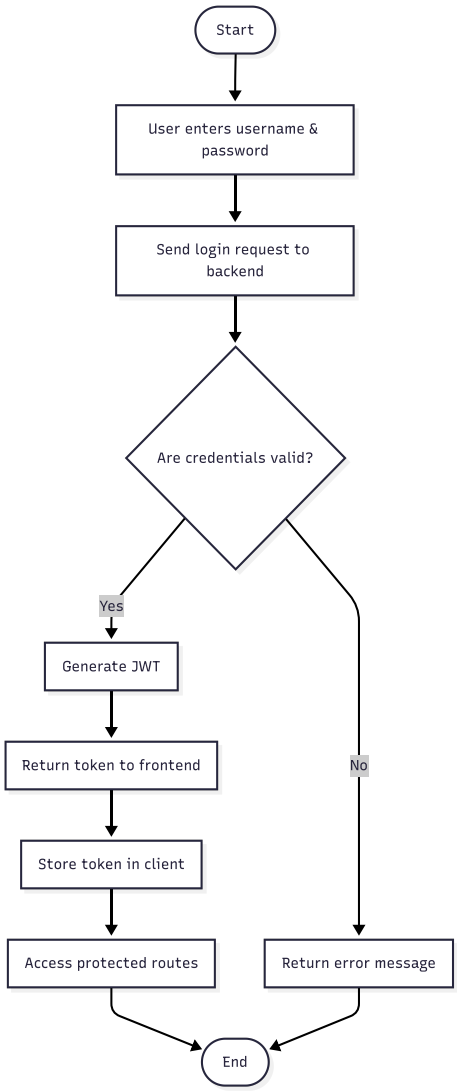
CONSTRAINT fk_checklist_user
FOREIGN KEY (user_id)
REFERENCES users(id)
ON DELETE CASCADE
);

-- ========================
-- RESOURCE TABLE
-- ========================
CREATE TABLE resource (
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
title VARCHAR(100) NOT NULL,
content TEXT,
category VARCHAR(50)
);
```

© 2020. Grand Canyon University. All Rights Reserved.

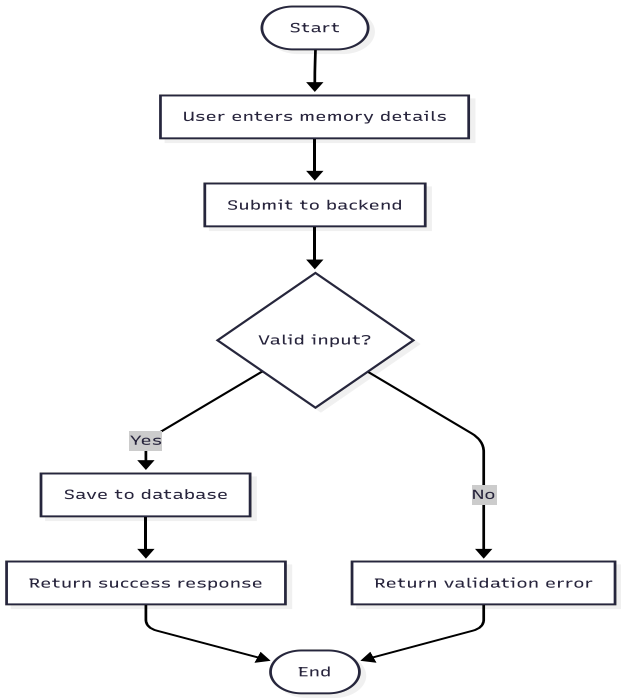
**Flow Charts/Process Flows:**

**Authentication Flow (Login)**

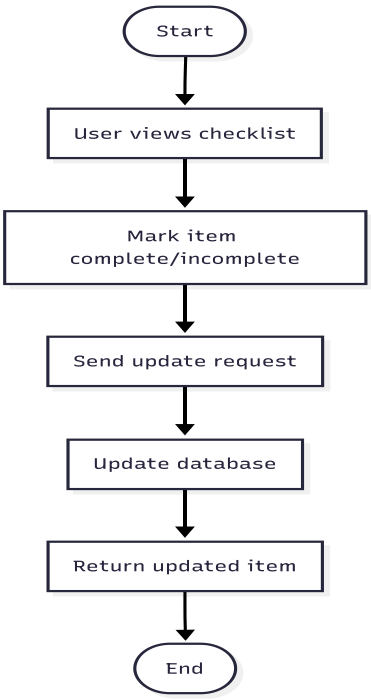



© 2020. Grand Canyon University. All Rights Reserved.

## Memory Creation Flow

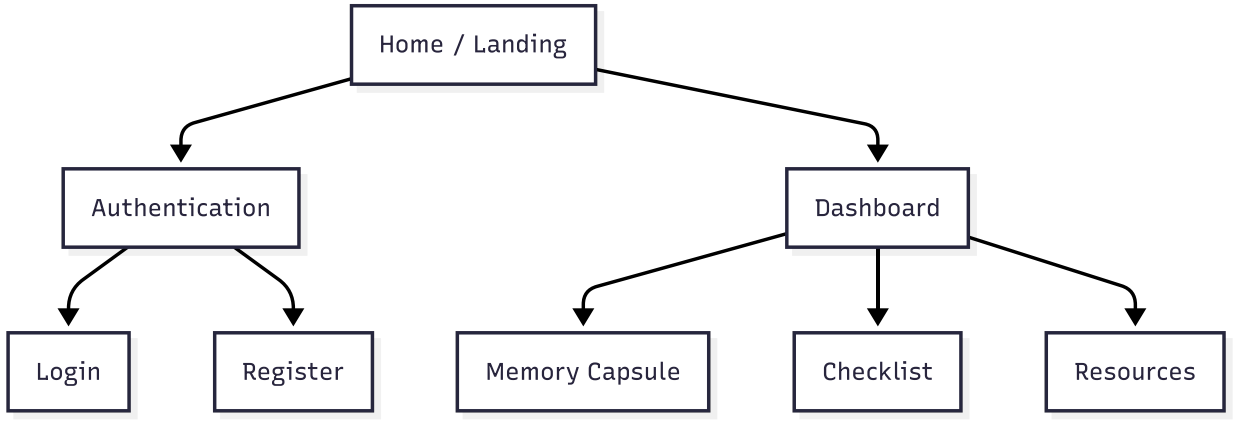



## Checklist Flow



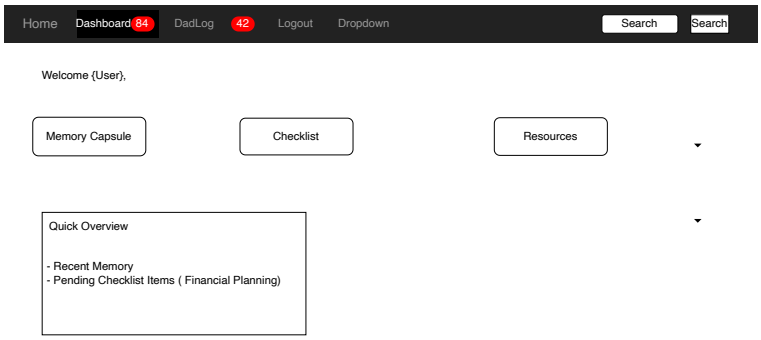

© 2020. Grand Canyon University. All Rights Reserved.

**Sitemap Diagram:**



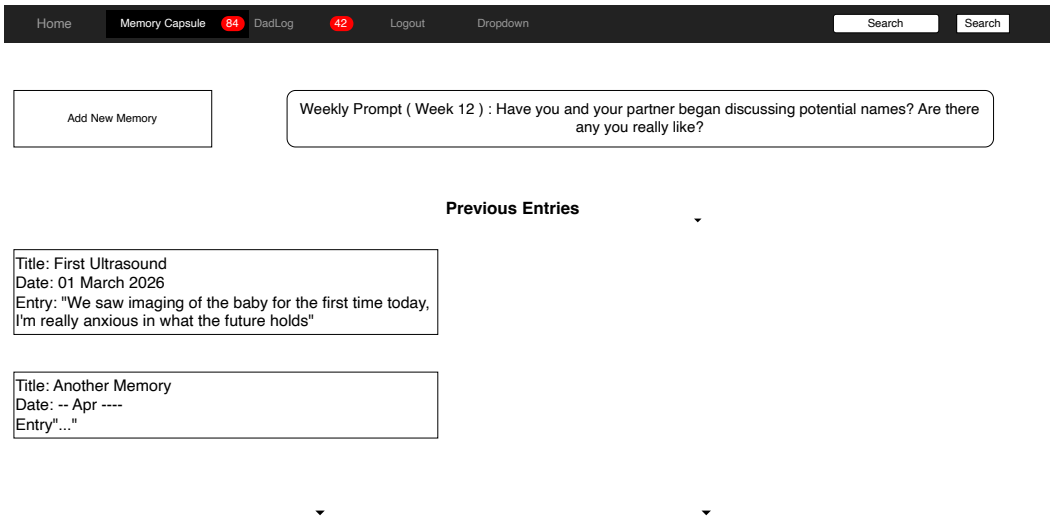

**User Interface Diagrams:**

**Dashboard**

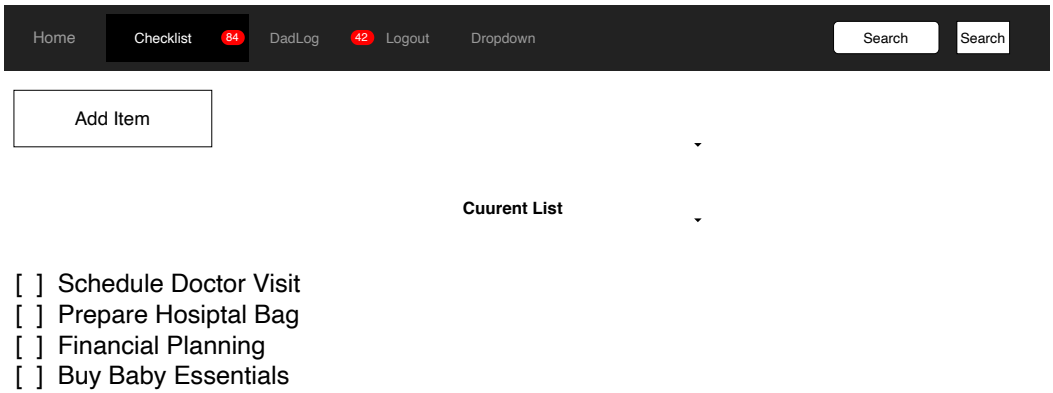



© 2020. Grand Canyon University. All Rights Reserved.

## Memory Capsule Page

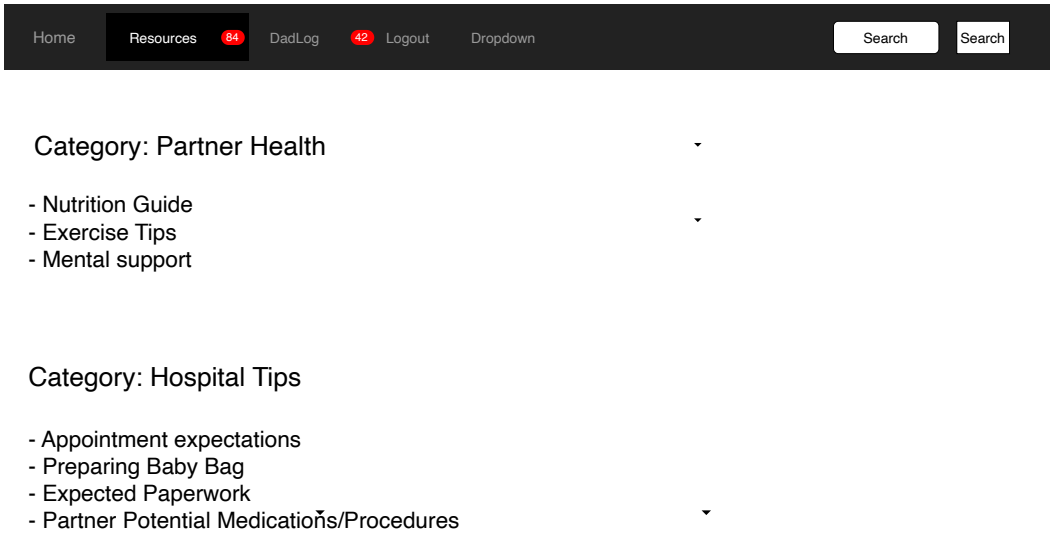



## Checklist Page

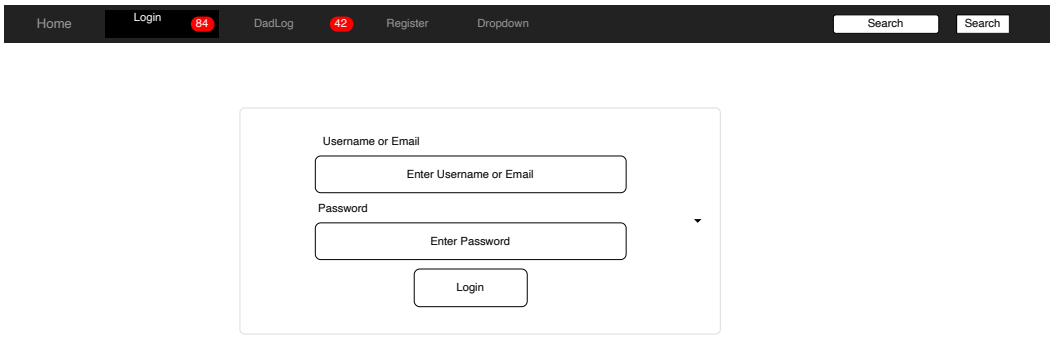



© 2020. Grand Canyon University. All Rights Reserved.

## Resources Page



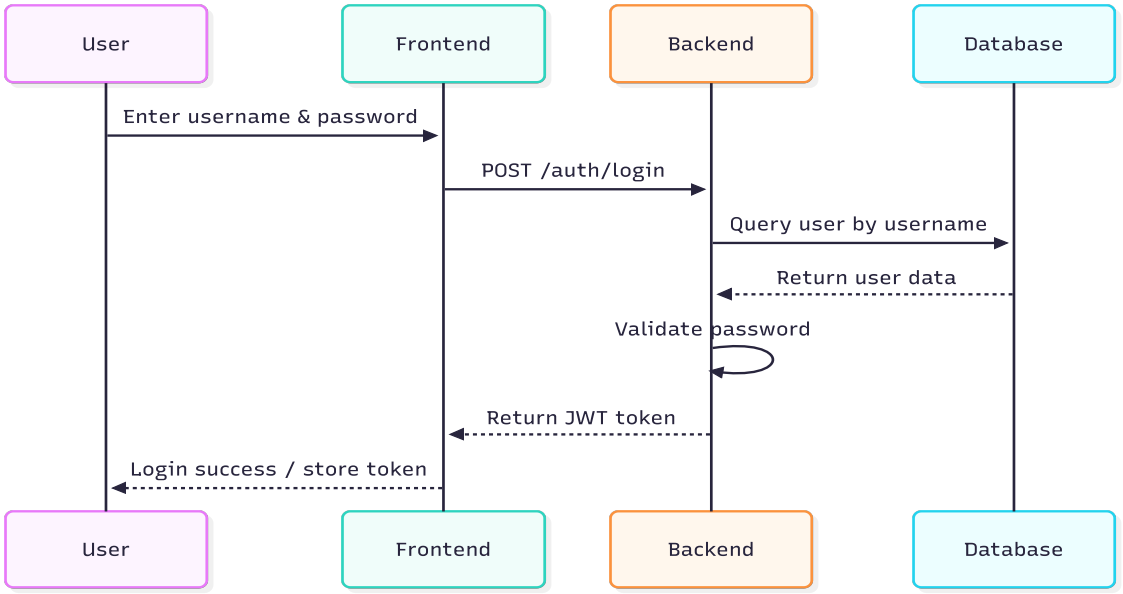

## Login



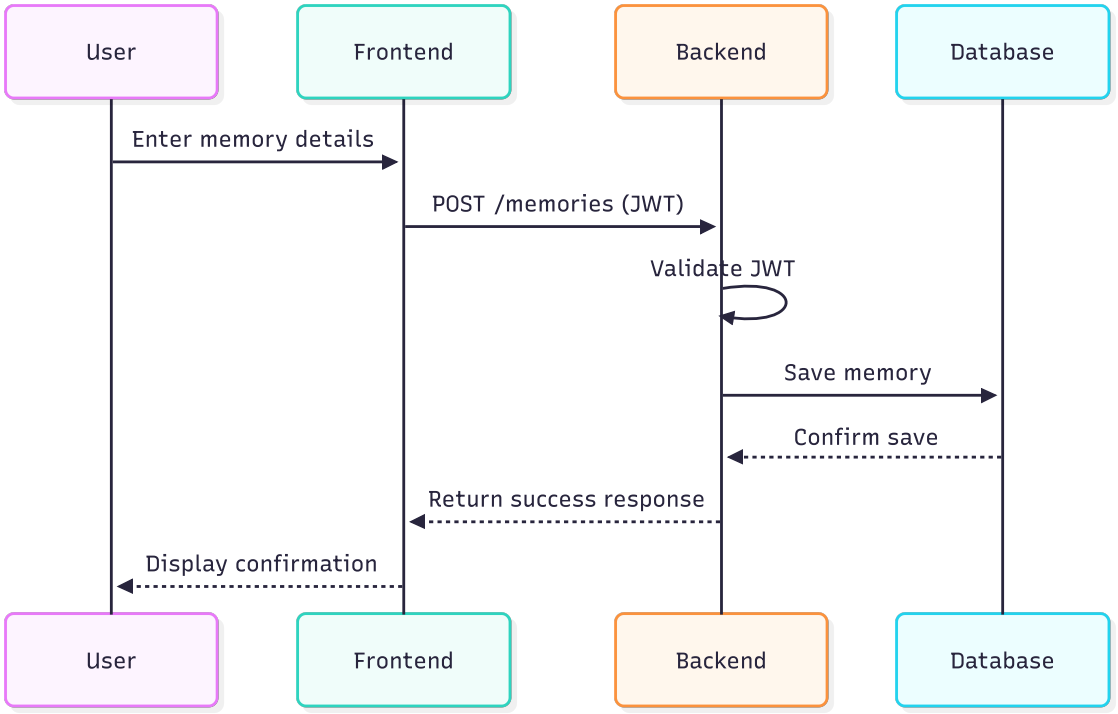

© 2020. Grand Canyon University. All Rights Reserved.

**UML Diagrams:**

**Login Sequence**




**Memory Creation Sequence**




© 2020. Grand Canyon University. All Rights Reserved.

UML Class diagrams are not included as the system design is already sufficiently represented through the ER diagram and layered architecture. The system is primarily API-driven, with relationships between entities captured in the database schema and system behavior represented using UML Sequence diagrams.

### Service API Design:

**Base URL (in development):** http://localhost:8080/

**Authentication:** Bearer <JWT_TOKEN>

---

**Auth Endpoints**

POST /auth/register - Registers a new user.

Request Body:

```
{
        username: "testuser",
        password: "password123"
}
```

Response:

```
{
        message: "User registered successfully"
}
```

---

POST /auth/login - Authenticates the user and returns a JWT.

Request Body:

```
{
        username: "testuser",
        password: "password123"
}
```

Response:

```
{
        token: "jwt_token_here",
        username: "testuser"
}
```

---

**User Endpoints**

GET /user/profile - Returns the authenticated user's profile.

**Headers:**
Authorization: Bearer <JWT_TOKEN>

Response:

```
{
        id: 1,
        username: "testuser",
        role: "USER"
}
```

© 2020. Grand Canyon University. All Rights Reserved.

## Memory Endpoints (Planned)

POST /memories - Creates a new memory entry.

**Request Body:**

```
{
	title: "First Ultrasound",
	content: "Today we saw the baby for the first time..."
}
```

Response:

```
{
	id: 1,
	title: "First Ultrasound",
	content: "Today we saw the baby for the first time...",
	created_at: "2026-04-15T10:00:00"
}
```

GET /memories - Retrieves all memory entries for the authenticated user.

**Response:**

```
{
	id: 1,
	title: "First Ultrasound",
	content: "Today we saw the baby...",
	created_at: "2026-04-15T10:00:00"
}
```

## Checklist Endpoints (Planned)

GET /checklist – Retrieves checklist items for authenticated users.

**Response:**

```
{
	id: 1,
	title: "Schedule doctor visit",
	is_completed: false
}
```

## Resource Endpoint (Planned)

GET /resources – Retrieves all resources – user authentication pending.

**Response:**

```
{
	id: 1,
	title: "Nutrition Guide",
	category: "Health"
}
```

© 2020. Grand Canyon University. All Rights Reserved.

**Third-Party APIs**

The system does not currently consume any third-party APIs. All services are internally developed and exposed through the application's REST API.

### NFR's (Security Design, etc.):

The DadPath system addresses key non-functional requirements, including security, performance, and scalability. Security is implemented with JWT-based authentication and BCrypt password hashing. Protected endpoints require valid tokens, and sensitive configuration data is stored in environment variables. Client-server communication is secured via HTTPS. Performance is achieved through HikariCP connection pooling for efficient database access and an Angular single-page application (SPA) that minimizes unnecessary page reloads and improves responsiveness. Scalability is supported by a stateless backend design, enabling the system to handle increased load and accommodate future feature expansion.

| Feature / Resource | Unauthenticated User | Authenticated User | Admin |
|---|---|---|---|
| Register Account | ✓ | ✓ | ✓ |
| Login | ✓ | ✓ | ✓ |
| Access Dashboard | X | ✓ | ✓ |
| View Memory Entries | X | ✓ (own data only) | ✓ |
| Create Memory | X | ✓ | ✓ |
| View Checklist | X | ✓ (own data only) | ✓ |
| Update Checklist | X | ✓ | ✓ |
| View Resources | ✓ | ✓ | ✓ |
| Manage Users | X | X | ✓ |

### Operational Support Design:

The Spring Boot backend employs standard logging (e.g., console logs) to track application events, errors, and API requests, facilitating debugging and resolving issues during development and deployment. Logging levels (INFO, WARN, ERROR) categorize system events, and logs can be configured per environment. The deployment platform (Railway) provides access to application logs, enabling real-time monitoring of system behavior. This approach ensures prompt error identification, tracking of system activity, and effective application maintenance as it scales.

### Other Documentation:

Because the project is in the planning and design phase, no additional supporting documentation is included in this section. The system's architecture, behavior, and implementation approach have already been thoroughly documented in architectural diagrams, flowcharts, ER diagrams, and API design specifications. These artifacts capture all the necessary details for development, making additional materials such as storyboards or task-tracking documents unnecessary at this stage.

© 2020. Grand Canyon University. All Rights Reserved.

## Appendix A – Technical Issue and Risk Log

| Issues and Risk Log | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Issue or Risk** | **Description** | **Project Impact** | **Action Plan/Resolution** | **Owner** | **Importance** | **Date Entered** | **Date to Review** | **Date Resolved** |
| I/R | What is the issue or risk? | How will this impact scope, schedule, and cost? | How do you intend to deal with this issue? | Who manages this issue? | | | | |
| R | Vercel backend CORS configuration complexity | Issues configuring cross-origin requests between Angular frontend and Spring Boot backend | Configure CORS policy in Spring Boot and test API calls incrementally during development | Andrew | High | 14 Apr 26’ | Ongoing | - |
| R | JWT storage security risks | Risk of insecure storage of JWT tokens on client side | Store token in secure client storage strategy and validate token on every request | Andrew | High | 14 Apr 26’ | Ongoing | - |
| R | Railway PostgreSQL connection limits | Limited number of concurrent database connections allowed by free tier | Implement HikariCP connection pooling and monitor usage during testing | Andrew | Medium | 14 Apr 26’ | Ongoing | - |
| R | Environment configuration mismatches | Differences between local, Vercel, and Railway environments | Use environment variables and separate dev/prod configuration files | Andrew | Medium | 14 Apr 26’ | Ongoing | - |

© 2020. Grand Canyon University. All Rights Reserved.

**Appendix B – References**


Angular. (2026). *Angular Documentation*. https://angular.dev

Angular Material. (2026). *Angular Material Documentation*. https://material.angular.dev

Bootstrap. (2026). *Bootstrap Documentation*. https://getbootstrap.com

Hibernate. (2026). *Hibernate ORM Documentation*. https://hibernate.org/orm/documentation/

JSON Web Token. (2026). *JWT Introduction*. https://jwt.io/introduction

PostgreSQL. (2026). *PostgreSQL Documentation*. https://www.postgresql.org/docs/

Spring Boot. (2026). *Spring Boot Documentation*. https://spring.io/projects/spring-boot

Spring Security Team. (2026). *Spring Security reference documentation*. https://docs.spring.io/spring-security/reference/index.html

Spring Security Team. (2026). *BCrypt*. *Spring Security API documentation.* https://docs.spring.io/spring-security/reference/api/java/org/springframework/security/crypto/bcrypt/BCrypt.html


© 2020. Grand Canyon University. All Rights Reserved. 

## Appendix C – External Resources

| | |
|---|---|
| **GIT URL:** | Not available at this stage. Version control repository will be established during the implementation phase. |
| **Hosting URL:** | Not available. Deployment will occur after development is completed in a later phase of the project. |

© 2020. Grand Canyon University. All Rights Reserved. 